

DISCLAIMER: ПЕРЕД НАЧАЛОМ НАСТОЯТЕЛЬНО РЕКОМЕНДУЕМ СКАЧАТЬ ДАННЫЙ ФАЙЛ В ФОРМАТЕ ПДФ

Тестовое задание — Full Stack Developer (OpenBrain)

Цель

Разработать небольшой MVP веб-приложения для управления заявками / запросами клиентов.

Что нужно реализовать

Backend

- REST API для работы с заявками
- Создание заявки
- Получение списка заявок
- Получение одной заявки по ID
- Обновление статуса заявки
- Валидация входных данных
- Обработка ошибок
- Подключение к базе данных

Frontend

- Страница со списком заявок
- Форма создания новой заявки
- Просмотр деталей заявки
- Фильтрация по статусу
- Понятный и аккуратный интерфейс

База данных

Минимум одна основная сущность:

- Request / Lead / Ticket

Пример полей:

- id
- title
- clientName
- phone / email
- description
- status
- createdAt

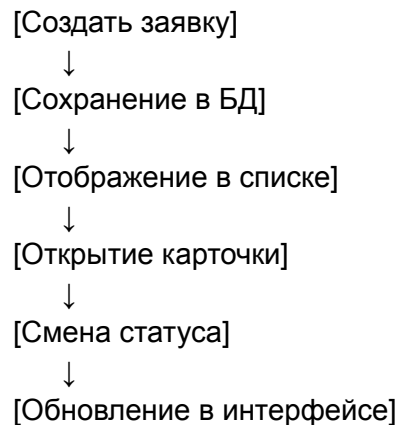
Дополнительно

- Docker
- README по запуску
- Postman collection или Swagger
- Нормальная структура проекта

Bonus

- Авторизация
- Роли
- Логирование
- Интеграция с mock API
- AI summary для описания заявки

A. Схема пользовательского потока



B. Пример структуры экранов

1. Login / вход (опционально)
2. Dashboard / список заявок
3. Create Request / форма создания
4. Request Details / карточка заявки
5. Filter by status / фильтр по статусу

С. Пример API-логики

POST /api/requests
GET /api/requests
GET /api/requests/:id
PATCH /api/requests/:id/status

Д. Пример статусов

New
In Progress
Waiting for Response
Completed
Rejected

Е. Визуальная архитектура

Frontend (React / Next.js)
↓
Backend API (Node.js / NestJS / Express)
↓
Database (PostgreSQL / MySQL)

Кандидат должен отправить:

1. GitHub repository

С понятной структурой проекта.

2. README

Где есть:

- что реализовано
- стек
- как запустить
- какие есть допущения
- что не успел сделать
- что бы улучшил в production-версии

3. Архитектурное описание

Очень коротко:

- почему выбрал такой стек
- как устроены модули
- как устроена БД
- как обрабатываются ошибки

4. Скриншоты интерфейса

Или короткое видео 2–4 минуты.

5. Деплой

Желательно, но не обязательно.

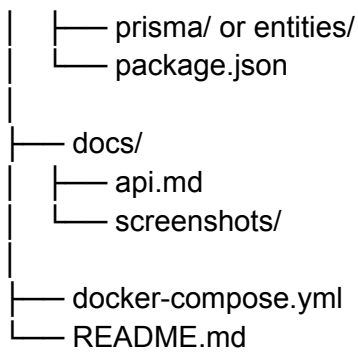
Оптимальная структура репозитория кандидата

Вариант 1 — монорепо

```
project-root/
├── client/
│   ├── src/
│   ├── components/
│   ├── pages/ or app/
│   └── package.json
├── server/
│   ├── src/
│   ├── controllers/
│   ├── services/
│   ├── routes/
│   ├── models/
│   ├── middleware/
│   └── package.json
├── docker-compose.yml
├── README.md
└── .env.example
```

Вариант 2 — Nest + React

```
project-root/
├── frontend/
│   ├── src/
│   ├── ui/
│   ├── pages/
│   └── package.json
├── backend/
│   ├── src/
│   ├── modules/
│   ├── common/
│   └── dto/
```



Нужен **чистый B2B UI**.

Минимум:

- аккуратная таблица
- понятная форма
- статусы в виде badge
- фильтр
- страница деталей
- адаптив хотя бы базовый

Хороший уровень:

- loading states
- empty states
- validation messages
- toast notifications
- приятная типографика
- логичная сетка

Плохой уровень:

- все на одной странице
- нет отступов
- нет состояний ошибок
- нет структуры
- грязные формы
- невозможно быстро понять сценарий

Критерии оценки — 100 баллов

1. Архитектура и структура проекта — 20 баллов

Оцениваем:

- понятное разделение frontend/backend
- логика по слоям
- читаемая структура папок
- масштабируемость

0–7 — хаос, все смешано

8–14 — рабочая, но средняя структура

15–20 — зрелое инженерное мышление

2. Backend и API — 20 баллов

Оцениваем:

- корректность REST API
- валидация
- обработка ошибок
- чистота кода
- работа с БД

0–7 — API слабый, много дыр

8–14 — все работает, но без глубины

15–20 — уверенный strong middle/senior уровень

3. Frontend и UX — 15 баллов

Оцениваем:

- удобство интерфейса
- логика экранов
- чистота компонентов
- состояния загрузки / ошибок

0–5 — слабый UI

6–10 — нормальный рабочий интерфейс

11–15 — хороший продуманный B2B frontend

4. Работа с БД и моделирование данных — 10 баллов

Оцениваем:

- нормальная схема
- типы полей
- статусы

- связи
- отсутствие лишней сложности

5. Качество кода — 15 баллов

Оцениваем:

- читаемость
- нейминг
- переиспользуемость
- отсутствие лишнего хардкода
- понятность решений

6. README и объяснение решения — 10 баллов

Оцениваем:

- можно ли быстро запустить
- понятно ли, что сделано
- честно ли описаны ограничения

7. Bonus / инженерная зрелость — 10 баллов

Сюда идет:

- Docker
- auth
- roles
- Swagger
- деплой
- логирование
- хорошая документация
- тесты

Быстрая шкала вердикта по кандидату

85–100 баллов

Однозначно сильный кандидат

Можно звать на финальное интервью почти сразу.

Признаки:

- хорошая структура
- зрелый backend
- чистый frontend
- понятный README
- видно product thinking
- есть инженерная аккуратность

70–84 балла

Хороший кандидат / strong middle

Стоит звать на техническое интервью.

Признаки:

- все основное сделано
- есть мелкие недочеты
- архитектура нормальная
- может быть полезен команде

55–69 баллов

Средний кандидат

Признаки:

- MVP работает
- но много слабых мест
- структура посредственная
- бизнес-логика поверхностная

Ниже 55

Не подходит

Карточка оценки кандидата

Имя кандидата:

Дата проверки:

Проверяющий:

1. Общий уровень

- Junior+
- Middle
- Strong Middle
- Senior

2. Итоговый балл

___/100

3. Оценка по блокам

- Архитектура: ___/20
- Backend/API: ___/20
- Frontend/UX: ___/15
- БД: ___/10
- Код: ___/15
- README/документация: ___/10
- Bonus: ___/10

4. Итоговый вывод

- Однозначно рекомендовать
- Рекомендовать с интервью
- Под вопросом
- Не рекомендовать

9) Что для OpenBrain особенно важно оценивать

С учетом вашей вакансии, я бы особенно смотрел на:

Обязательно

- API-мышление
- валидацию
- статусы и бизнес-логику
- аккуратную БД
- чистую структуру

- реальную работоспособность

Очень важно

- умеет ли кандидат объяснить, почему сделал именно так
- умеет ли не переусложнять
- понимает ли, как строятся B2B/internal tools

Менее важно

- супер-красивый UI
- идеальная анимация
- 100% production polish

Критерии оценки:

Мы оцениваем не только факт выполнения задания, но и качество инженерного мышления кандидата. Для нас важны: структура проекта, чистота кода, логика API, корректность работы с данными, понятность интерфейса, обработка ошибок, качество README и способность объяснить свои технические решения. Наличие bonus-функций приветствуется, но не является обязательным.

Что важно для нас

Нам не нужен “идеальный enterprise-продукт”. Мы хотим увидеть, как вы думаете как инженер: как проектируете архитектуру, как организуете код, как строите API, как работаете с данными и как принимаете технические решения в рамках реального MVP.